

## **FEASIBILITY STUDY**

### **Smart Attendance and Classroom Access Platform with Face Verification and Geofenced Check-In**

PID 12 | Group 27

Mentor: Mr. Pasan Sarathchandra

**12/07/2026**

Name : Withanage M.H.  
Index No : 230736R

## TABLE OF CONTENTS

|                                                        |           |
|--------------------------------------------------------|-----------|
| <b>Individual Contributions.....</b>                   | <b>3</b>  |
| <b>1. Introduction.....</b>                            | <b>4</b>  |
| 1.1 Overview of the Project.....                       | 4         |
| 1.2 Objectives of the Project.....                     | 4         |
| 1.3 The Need for the Project.....                      | 4         |
| 1.4 Overview of Existing Systems and Technologies..... | 5         |
| 1.5 Scope of the Project.....                          | 5         |
| 1.6 Deliverables.....                                  | 6         |
| <b>2. Feasibility Study.....</b>                       | <b>7</b>  |
| 2.1 Financial Feasibility.....                         | 7         |
| 2.2 Technical Feasibility.....                         | 8         |
| 2.3 Resource and Time Feasibility.....                 | 10        |
| 2.4 Risk Feasibility.....                              | 11        |
| 2.5 Social/Legal Feasibility.....                      | 14        |
| <b>3. Considerations.....</b>                          | <b>16</b> |
| <b>4. References.....</b>                              | <b>18</b> |

## Individual Contributions

| Member                               | Contribution                                                                                                                                            |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| WICKRAMAARACHCHI W.A.A.U.<br>230702K | Scope of the Project<br>Financial Feasibility<br>Resource and Time Feasibility<br>Final review and formatting                                           |
| WIJAYARATHNE T.W.O.A.<br>230714A     | Objectives of the Project<br>Overview of Existing Systems and Technologies<br>Technical Feasibility<br>Considerations<br>Final review and formatting    |
| WITHANA W.D.P.S.<br>230734J          | Overview of the Project<br>Overview of Existing Systems and Technologies<br>Risk Feasibility<br>Social/Legal Feasibility<br>Final review and formatting |
| WITHANAGE M.H.<br>230736R            | The Need for the Project<br>Deliverables<br>Financial Feasibility<br>Considerations<br>Final review and formatting                                      |

## **1. Introduction**

### **1.1 Overview of the Project**

This project proposes a web and mobile-based secure attendance management platform for universities. The platform addresses the limitations of existing manual and basic digital attendance tracking systems by combining face verification and geofenced location validation to prevent proxy attendance and improve reliability of attendance records.

Lecturers create and manage attendance sessions through a web application, while students mark their attendance through a mobile application. Each attendance check-in verifies the student's identity and location. The system benefits lecturers by reducing administrative overhead and providing real-time attendance visibility, and benefits students by providing a transparent and accessible record of their own attendance. Also benefits administrators by centralizing attendance data and reporting.

*Inputs:* Student registration details and facial images sourced from existing university student records, facial images captured during attendance check-in, device location data for geofence validation and dynamic QR code scanning for classroom presence confirmation, lecturer defined class sessions and timetables, and administrator configured geofence boundaries for classrooms or lecture locations.

*Outputs:* Verified attendance records, live attendance status during active sessions, student attendance history, attendance reports and analytics, and notifications for attendance related alerts and upcoming classes.

### **1.2 Objectives of the Project**

- Design and implement a mobile and web-based attendance management system.
- Allow lecturers to create, activate, and close attendance sessions for lectures.
- Allow students to mark attendance through a mobile application.
- Verify student identity using backend face verification based on face embeddings.
- Reduce proxy attendance using face verification and location-based validation.
- Generate attendance reports for lecturers, administrators, and students.
- Provide a centralized system to manage students, lecturers, courses, and attendance records.

### **1.3 The Need for the Project**

Attendance is an important part of academic management in universities and training institutions. However, many institutions still use paper-based or basic digital attendance methods. These methods can be slow, error-prone, and difficult to manage when there are large student groups.

A smart attendance system is useful because it saves lecture time, reduces manual work, improves accuracy, and provides better access to attendance records. It also helps lecturers identify low attendance early and allows lecturers and administrators to generate reports without manually checking attendance sheets. Since attendance fraud such as proxy marking can occur in conventional systems, identity verification and location validation are essential for improving the reliability of attendance records.

However, single-factor verification methods also have limitations. GPS-only attendance can be unreliable indoors, QR code-only attendance can be shared with absent students, and face verification alone cannot confirm that a student is physically present in the classroom.

Therefore, combining face verification with geofenced location validation and dynamic QR code verification makes the system more reliable.

### 1.4 Overview of Existing Systems and Technologies

Jibble and Truein are employee-based attendance systems that use face recognition with GPS tracking and geofencing. However, these systems are mainly useful for field staff and multi-site environments [1],[2].

Quickly Attendance and SEAtS ONE are more related to educational institutions. They support methods such as QR code scanning, location confirmation, Bluetooth beacons, Wi-Fi, and card based check-in for student attendance [3],[4].

AttenFace is a research-based classroom attendance system that uses classroom camera snapshots and face recognition [5].

A closely related study by Babu et al. [6] combines geofencing with facial recognition. GPS location is validated before a facial embedding (via face\_recognition) matches a stored reference, on Django/MySQL backend. Liveness detection is not yet implemented and noted as future work.

This project adopts the same dual-verification principle, with location validation via GPS-based geofencing and a time-limited, dynamically refreshing QR code.

#### Planned Technology Stack:

Web Application - Next.js and TypeScript [7]

Student Mobile Application - React Native with Expo [8]

Backend - Python FastAPI [9]

Database - PostgreSQL [10]

Geofenced Location Validation - GPS coordinate and radius-based validation

Face Embedding Extraction - DeepFace [11] using a pretrained ArcFace model [12]

Similarity Comparison - Cosine similarity with a configurable verification threshold [13]

Face Landmark Detection and Liveness Check - Google ML Kit face detection API [14]

Image Preprocessing - OpenCV [15]

All selected tools and libraries are open-source or freely available. Next.js, React Native with Expo, FastAPI, and PostgreSQL are widely used, well documented frameworks that reduce implementation risk. Deepface with a pretrained ArcFace model enables face embedding extraction without custom model training. Google ML Kit face detection provides on-device face detection, facial landmarks, head orientation angles, and eye open probabilities [14]. These outputs will be used to implement a basic randomized challenge response liveness check, such as asking the students to blink, turn their head left or right. PostgreSQL provides reliable relational storage for attendance records, session data, and verification logs.

### 1.5 Scope of the Project

#### *Students* (Mobile Application)

- Log in and maintain a personal profile.
- View personal profile, timetable, and attendance history.
- Mark attendance by performing face verification and location validation.
- Receive notifications for upcoming classes and attendance related alerts.

#### *Lecturers* (Web Application):

- Log into the system and maintain a personal profile.
- Create and manage courses, attendance sessions, and timetables.
- View live attendance status during active sessions.
- Manually mark attendance for verification failed students.

- Generate attendance reports and monitor students attendance analytics.

*Administrators (Web Application):*

- Create and manage student and lecturer accounts.
- Configure classrooms, geofence locations, and system settings.
- View university wide attendance reports and analytics.
- Manage courses and academic information.

*Main Functions of the System*

- User authentication with role based access control.
- Face verification during attendance check-in.
- Geofenced and dynamically refreshing QR code based location validation to confirm classroom presence.
- Attendance session management.
- Attendance history, reporting, and analytics.
- Secure storage of attendance records and verification logs.

## **1.6 Deliverables**

A web-based attendance management system and a mobile application that enable secure attendance management for lecturers, administrations, and students through face verification and geofenced check-ins. The system will provide intuitive graphical user interfaces, a secure backend with REST APIs and authentication, attendance reporting and analytics dashboards at university, class, and individual student level, and an AI-based face verification module built on a pretrained face embedding model, integrated into the check-in workflow.

The project will also deliver technical documentation including the system architecture, database design, API design, and AI-based face verification workflow to support deployment, maintenance, and future enhancements.

## 2. Feasibility Study

### 2.1 Financial Feasibility

The financial feasibility of the Smart Attendance and Classroom Access Platform is assessed by considering the costs of software development, cloud infrastructure, data storage, mobile application distribution, and future enhancement costs.

The selected technology stack reduces initial expenditure because the main development frameworks and libraries are open source or available without licence fees. The development and testing devices are already available to the development team. Since the system primarily uses software based solutions, no immediate expenditure is expected for additional programming equipment during initial implementation.

The principal operational costs are associated with hosting the backend services, PostgreSQL database, Redis service, object storage, web dashboard, and mobile application build and distributional services. For the initial deployment, the FastAPI backend, Redis, and background workers are expected to run on an Amazon EC2 instance using available AWS Free Tier credits. New AWS customers receive USD 100 in credits at sign up and may earn up to an additional USD 100 by completing selected AWS activities [16]. The AWS Free Tier is available for up to six months or until the available credits are exhausted, whichever occurs first [17].

This can reduce the initial backend hosting cost. However, it should not be considered as a permanent no cost production solution. Resource usage must be monitored because continuous backend operation and computationally intensive face verification may consume the available credits quickly. A larger EC2 instance may also be required when the number of concurrent check-ins increases.

PostgreSQL will initially be hosted using the Supabase Free Plan, which provides limited resources suitable for development and initial evaluation [18]. A paid managed database plan should be considered for regular institutional use, particularly when automated backups, increased storage, and improved recovery support are required.

Cloudflare R2 will be used to store temporary check-in images and other required files. R2 provides a monthly free allowance for storage and operations, after which charges are based on usage [20]. A short retention policy will also be applied so that temporary images are deleted after verification or manual review, reducing unnecessary storage growth and helping the system remain within the available free tier limits.

The lecturer and administrator web dashboard frontend, developed using Next.js and TypeScript will be deployed on Vercel. The free Hobby plan can support development and limited evaluation [21]. A paid plan or another hosting option may be considered for production deployment if higher capacity or operational support is required.

Expo Application Services will be used to build and distribute the student mobile application, which will be developed using React Native with Expo. The free plan provides limited build and update capacity suitable for initial deployment and testing, while paid plans may be required for frequent production builds, larger usage limits, and application store distribution [22].

| <b>Cost Item</b>                                                          | <b>Projected Cost/year (LKR)</b> |
|---------------------------------------------------------------------------|----------------------------------|
| Salaries and wages                                                        | 0.00                             |
| Programming equipment                                                     | 0.00                             |
| Software licences                                                         | 0.00                             |
| Development utilities and tools                                           | 0.00                             |
| FastAPI backend, Redis and background workers hosting (AWS EC2 Free Tier) | 0.00*                            |
| PostgreSQL database hosting (Supabase Free Plan)                          | 0.00*                            |
| Object Storage (Cloudflare R2 Free Plan)                                  | 0.00*                            |
| Web dashboard frontend hosting (Vercel Hobby Plan)                        | 0.00*                            |
| Mobile Application build and distribution (Expo Free Plan)                | 0.00*                            |
| <b>Total projected initial annual cost</b>                                | <b>0.00*</b>                     |

\* The projected cost remains zero only while usage stays within the applicable free tier limits and cloud credits. Charges may apply when compute, storage, database, build, or operational limits are exceeded.

Overall, the proposed system is financially feasible because it can be initially deployed with low infrastructure expenditure. The selected free plans and cloud credits reduce implementation and early operational costs. However, the adopting institution should allocate a recurring budget for reliable production hosting, database backups, storage, and future scaling.

## 2.2 Technical Feasibility

The technical feasibility of the proposed Smart Attendance and Classroom Access Platform is assessed based on the practicality of the selected technical solutions, the maturity of the technologies, the availability of required development resources, and the ability of the system to meet performance, security, and reliability requirements. The project adopts modern, scalable, and mostly free and open-source technologies within a service-oriented modular architecture to support reliable institutional deployment. The system requires a student mobile application, a lecturer and administrator web application, a secure backend, a relational database, and verification components for face verification, geofence validation, and dynamic QR-based classroom confirmation.

For the student mobile application, React Native with Expo will be used. Flutter was also considered because it is a strong cross-platform mobile development framework and can provide good performance with a consistent user interface across Android and iOS. However, React Native with Expo is selected because it provides faster development for this project, easier integration with JavaScript and TypeScript-based frontend development, and strong support for mobile features such as camera access, location services, QR scanning, and push notifications [8],[23]. Expo also simplifies mobile development, testing, and device feature

integration, which is useful for building and validating the student application across different devices.

For the backend, Python FastAPI will be used to develop the core API layer of the system. Spring Boot, Node.js with Express or NestJS, and Django were also considered as possible backend technologies. Spring Boot is strong for enterprise-level systems, while NestJS provides a structured backend framework for TypeScript-based development. Django also provides a mature Python-based web framework with built-in features. However, FastAPI is selected because it provides high-performance REST API development, strong request validation through Pydantic, automatic OpenAPI documentation, and good support for modular backend design [9]. It is also highly suitable for this project because the face verification and image preprocessing components are Python-based. Using FastAPI allows the backend to integrate more naturally with OpenCV, DeepFace, and ArcFace based embedding extraction, and other computer vision libraries without requiring separate language bridges.

For data storage, PostgreSQL will be used as the primary database. Other options such as MySQL, MongoDB, and Firebase Firestore were also considered. MySQL is a reliable relational database, MongoDB provides flexible document-based storage, and Firebase can simplify mobile application development with managed backend features. However, PostgreSQL is selected because the system contains highly structured and relational data. These entities have clear relationships, and attendance records require data consistency, constraints, and reliable transaction handling. PostgreSQL supports relational integrity, indexing, transactions, role based database access, and long-term structured data storage [10], making it well suited to the system's data management requirements.

For the face verification component, a pretrained face embedding model will be used instead of training a new model from the beginning. Options such as FaceNet, ArcFace, InsightFace were considered. Cloud-based face recognition APIs were also considered, but they are not preferred because they introduce privacy concerns, recurring cost, vendor dependency, and less control over biometric-related data. The selected approach is to use DeepFace with a pretrained ArcFace model for backend face verification. ArcFace is suitable because it is widely used for face recognition tasks and produces strong face embeddings that can be compared for identity verification [12]. DeepFace provides a practical Python-based interface for using pretrained face recognition models, which makes it easier to integrate with the FastAPI backend and OpenCV-based preprocessing pipeline [11]. This approach is technically feasible because it avoids the high cost and complexity of collecting a large training dataset and training a custom face verification model. It also keeps the verification process controlled within the backend, which improves consistency, maintainability, and security across different student devices.

For image preprocessing, OpenCV and DeepFace will be used to prepare captured face images before embedding extraction. The mobile application will capture the student's face image and send a compressed image to the backend. The backend will then perform the main preprocessing steps such as face detection, face cropping, resizing, alignment where supported, and basic image quality checking. This preprocessing step is important because face verification accuracy can be affected by poor lighting, blurred images, wrong face angles, or multiple faces in the image [11],[15]. The system should reject images where no face is detected, more than one face is detected, or the image quality is too poor for reliable verification. By performing preprocessing in the backend, the system can maintain consistent verification behavior across different student devices.

For location and classroom presence validation, the system will use a combination of GPS-based geofencing and dynamic QR code verification. GPS-only verification was considered, but it is not reliable enough to confirm exact classroom presence because GPS accuracy can be weak inside buildings. This combined approach is technically feasible because it does not require additional classroom hardware such as Bluetooth beacons, RFID readers, or UWB devices. It also provides a practical balance between implementation complexity, cost, and classroom-level attendance verification.

For liveness support, Google ML Kit Face Detection will be used in the student mobile application for basic live-capture checks [14]. A backend video-based liveness approach was considered, but uploading video clips during attendance can increase network usage, backend processing load, and check-in delay, especially when many students mark attendance at the same time. The selected approach is technically feasible because it improves protection against simple photo-based spoofing while keeping the attendance flow practical for real classroom use. However it is a basic liveness detection mechanism, not a complete defense against advanced spoofing attacks.

Redis will be used as a supporting in-memory data store for short-lived and performance-sensitive operations. Although PostgreSQL can store the system's permanent data, using PostgreSQL alone for every temporary validation check may create unnecessary database load during peak attendance periods. Redis will therefore be used for temporary data such as dynamic QR token states, active session caching [24], rate-limiting counters [25], login attempt counters, and queued face verification tasks [26]. During peak check-in periods, Redis may temporarily hold verification jobs that are processed by background workers at a controlled rate [26]. Redis will not replace the primary database. Permanent and transactional data such as users, courses, attendance records, verification results, manual overrides, and audit logs will be stored in PostgreSQL. This separation allows PostgreSQL to provide reliable long-term data storage, while Redis improves the performance and responsiveness of time-sensitive attendance operations.

## **2.3 Resource and Time Feasibility**

The required resources for the proposed system include software resources, programming equipment, human resources, mobile testing devices, and hosting resources.

The software resources include React Native with Expo for the mobile application, Next.js and TypeScript for the web application, FastAPI for the backend, PostgreSQL for the database, OpenCV and DeepFace with ArcFace for face verification, and Google ML Kit for mobile side face detection and liveness check. Almost all of these technologies are open-source or freely available.

The programming equipment required for development includes laptops or desktop computers, which are already available to the development team. Since the system uses pretrained face verification models and software based solutions, expensive hardware is not required. However, various models of Android and iOS mobile devices are needed to test device compatibility for camera access, GPS validation, QR scanning, and face detection features. This can be done by testing the system among the batchmates, as there is sufficient mobile device diversity among them.

The project also requires skilled human resources to develop the mobile application, web dashboard, backend APIs, database, AI-based check-in workflow, testing, and

documentation. The development team consists of four members, which is sufficient to divide the workload across these areas.

The required hosting resources are available through the cloud platforms identified in the financial feasibility section. These services provide sufficient compute, storage, web deployment, and mobile build capacity for the planned implementation. Higher capacity paid plans can be adopted if system usage or reliability requirements increase.

Therefore, the project has the required resource feasibility. The project is also time feasible if development follows the planned phased schedule with continuous testing throughout the project timeline. The work will progress from requirements and system design to core platform development, attendance verification integration, testing, deployment, and the final demonstration.

### **Milestones:**

- Week 1-2: Requirements elicitation, requirements analysis, project scope finalization, project proposal, feasibility report, and project schedule preparation.
- Week 3-4: SRS preparation, system architecture design, security design, API design, PostgreSQL and Redis database design, web and mobile UI design, and test strategy planning.
- Week 4-6: Backend project setup, database setup, CI setup, initial test setup, authentication, user management, and basic system foundation.
- Week 5-10: Development of the core platform, including the web dashboard, student mobile application, course and timetable management, attendance session management, reports, and notifications.
- Week 6-10: Development of attendance verification components, including image preprocessing and enrollment, face verification, active liveness detection, geofence validation, and dynamic QR validation.
- Week 4-10: Continuous test-driven development, automated unit testing, module integration, and API integration throughout implementation.
- Week 11-12: End-to-end testing, security testing, performance testing, device compatibility testing, and staging deployment.
- Week 12-13: System usability testing, regression fixes, and final adjustments based on testing feedback.
- Week 13-14: Final deployment, release APK preparation, final demonstration, final report, and user guide completion.

### **2.4 Risk Feasibility**

The proposed Smart Attendance and Classroom Access Platform involves technical, operational, security, privacy, and resource related risks. This section identifies the key risks and the mitigation strategies planned to reduce their impact on the system and its users.

- Privacy concerns over biometric and location data  
Since the system processes facial data and student location data for attendance verification, it will create legal and ethical privacy concerns if the data is not handled carefully. This concern will be addressed by obtaining informed consent from students, clearly explaining how facial and location data are used, and storing only the minimum data required for verification, such as reference face embeddings, attendance location metadata, and necessary verification logs. A defined data retention policy will also be applied so that temporary check-in images, location records, and verification logs are not retained permanently.

- **Face verification failure due to poor lighting**  
Lecture halls in many local universities may not have consistent lighting conditions, which could cause the face verification step to fail for legitimate students. To mitigate this, the captured image will be checked before verification. ML Kit's face detection will confirm a face present, centered, and clearly visible, while a separate brightness and blur check performed using OpenCV will flag images that are too dark or unclear. If a check fails, the system will allow the student to retry the capture, and repeated failed attempts will be routed to the lecturer for manual review rather than blocking the student outright.
- **Location and QR validation risks**  
GPS signals can be inaccurate inside multistorey university buildings, while a dynamic QR code may still be shared during its short validity period. These risks will be reduced by requiring GPS-based geofence validation together with a short-lived QR token. Borderline cases where the GPS result is close to the permitted location boundary may be reviewed and manually approved by the lecturer where appropriate.
- **Photo or video spoofing**  
Students may attempt to use photos or recorded videos instead of a live face to mark attendance for another person. The mobile application will use basic liveness checks and image quality validation to reduce these spoofing risks. Suspicious attempts can be logged for lecturer review.
- **High check-in traffic and slow face verification response**  
When many students check in within a short time, such as at the start of a lecture, the backend's attendance and face verification modules may become overloaded, causing slower verification responses and a poor user experience. This issue will be managed by using a controlled check-in window so students are not required to submit at the exact same moment. Student reference face embeddings will be precomputed and stored in advance, and check-in images will be resized and compressed into JPEG format before backend processing to reduce processing load. Face verification requests can also be placed in a Redis based request queue and processed by background workers in a controlled manner during peak check-in periods and prevent system overload [26].
- **Network failure during a lecture**  
Poor or interrupted network connection during a lecture could prevent students from completing check-in or QR validation on time. Therefore the system will provide a limited retry window for attendance actions when connectivity issues occur. If a student is unable to complete the process within the retry window due to network failure, the lecturer can review the case and manually update the attendance status when appropriate.
- **Outdated reference photos**  
If a student's photo in the existing university records was taken several years ago, changes in appearance may cause the face verification to reject a legitimate student during check-in. This risk will be addressed by running a test verification period before full rollout, so students can identify possible verification issues early. If a student repeatedly fails face verification due to an outdated reference photo, they will be directed to update their photo through the official university photo update process. Until the issue is resolved, lecturers can manually review and mark attendance for verification-failed students.

- **Database failure**  
A database failure could affect attendance records, user data, and verification logs. This risk will be managed by hosting PostgreSQL on the Supabase Free Plan during the initial deployment, together with data validation, transaction handling, and periodic manual backups to maintain off-site backup copies [18],[19]. Since Supabase provides automated daily backups for Pro, Team, and Enterprise Plan projects, a paid managed PostgreSQL plan can be considered to improve automated backup handling, reliability, and recovery support [18].
- **Incorrect manual approval by a lecturer**  
Lecturers can manually update attendance when verification issues occur. To prevent error or misuse, each manual update will require a reason and will be recorded in a database audit log with details such as lecturer ID, student ID, session ID, status change, reason, and timestamp. This allows administrators to review manual changes when needed.
- **Device compatibility issues**  
Some students may use older mobile devices with lower specifications that cannot reliably support the camera, ML Kit based face detection, liveness checks, GPS access, or QR scanning features. This issue will be handled by defining minimum device requirements for the mobile app, as well as testing the application on different Android and iOS devices. If a student's own device is not supported, they may use a friend's compatible device as a fallback, by logging into their own account and completing the check-in themselves. If the verification process still cannot be completed, the lecturer can review the case and manually update when appropriate.
- **Open source dependency and licensing risk**  
The system uses several open-source libraries and frameworks such as FastAPI, React Native, PostgreSQL, Redis, OpenCV and DeepFace. Although these technologies reduce development cost and vendor dependency, there is a risk that some technologies may change their license terms, become unsupported, introduce breaking changes or contain security vulnerabilities in the future.  
This risk cannot be completely eliminated. However it can be managed by selecting actively maintained libraries and keeping alternative libraries where possible. If a critical dependency requires stronger long-term support, commercially supported open-source alternatives or paid support plans can be considered without changing the overall open-source-first architecture.
- **Free Tier platform limitations and pricing change risk**  
Free tier limits, service availability, or pricing may change and increase operational costs. This risk will be mitigated by designing the system to be containerized and portable so that it can be migrated to another platform when required. The adopting institution should also allocate a recurring budget to reduce the impact of such changes.
- **Team or development resource unavailability**  
The project schedule may be affected if a team member becomes temporarily unavailable due to illness or personal emergencies. Development resources may also be affected by issues such as laptop failure, device loss, or limited access to mobile testing devices. These risks will be managed by using GitHub as a version control system, maintaining backup copies of important files, sharing project knowledge among team members, and distributing tasks so that another member can continue the

work if one member or device becomes temporarily unavailable. As mentioned in the resource and time feasibility section device testing needs can be met by testing the system among batchmates.

### **Risk Assessment Summary**

| <b>Risk Description</b>                                   | <b>Impact Severity</b> | <b>Likelihood of Occurrence</b> | <b>Overall Risk Severity</b> |
|-----------------------------------------------------------|------------------------|---------------------------------|------------------------------|
| Privacy concerns over biometric and location data         | High                   | Medium                          | High                         |
| Face verification failure due to poor lighting            | High                   | Medium                          | High                         |
| Location and QR validation risks                          | High                   | High                            | High                         |
| Photo or video spoofing                                   | High                   | Medium                          | High                         |
| High check-in traffic and slow face verification response | Medium                 | Medium                          | Medium                       |
| Network failure during a lecture                          | Medium                 | Medium                          | Medium                       |
| Outdated reference photos                                 | Medium                 | Medium                          | Medium                       |
| Database failure                                          | High                   | Low                             | Medium                       |
| Incorrect manual approval by a lecturer                   | Medium                 | Low                             | Medium                       |
| Device compatibility issues                               | Medium                 | Medium                          | Medium                       |
| Open source dependency and licensing risk                 | Medium                 | Medium                          | Medium                       |
| Free Tier platform limitations and pricing change risk    | Medium                 | Medium                          | Medium                       |
| Open source dependency and licensing risk                 | Medium                 | Medium                          | Medium                       |
| Team or development resource unavailability               | Medium                 | Medium                          | Medium                       |

### **2.5 Social/Legal Feasibility**

The Smart Attendance and Classroom Access Platform is socially and legally feasible, but it must handle student biometric and location data carefully. Since the system uses facial verification and GPS based geofence validation, privacy, consent, and institutional policy compliance are important considerations.

Facial data and location data are sensitive because they are linked to student identity and attendance records. As mentioned in the privacy concern risks, the system will collect only the minimum data required for verification and images captured during check-in will not be retained permanently. Also, access to student records will be controlled using role based access control for students, lecturers, and administrators.

Students should be clearly informed about what data is collected, why it is collected, how it is used, and how long it will be retained. Consent should be obtained during first time account activation in the test verification period, before facial and location data are used for attendance verification. The system should also ensure that location data is collected only during active attendance actions, such as check-in or QR validation, rather than continuously tracking students.

The system must also avoid unfair attendance decisions. Verification failures may occur due to poor lighting, outdated reference photos, device issues, GPS inaccuracy, and network connectivity issues. Therefore, as mentioned in the risk feasibility section, retry options and lecturer review will be provided for valid failed or uncertain cases. Manual attendance updates will be recorded in a database audit log to maintain accountability.

The system should be designed and developed according to university IT policies and student data policies, as universities in Sri Lanka generally have their own rules for handling student records and academic systems. Before deployment, the final system should also be reviewed to confirm that it complies with the policies of the institution adopting it. Third party technologies Next.js, React Native, FastAPI, PostgreSQL, OpenCV, DeepFace, ArcFace, and Google ML Kit should be checked for licensing to confirm there are no restrictions preventing deployment in a university environment.

Overall, the proposed system is socially and legally feasible if informed consent, limited data collection, secure access control, defined data retention, lecturer review, and institutional policy compliance are followed.

### 3. Considerations

#### 1. Accuracy and Reliability

Accuracy refers to the correctness of the attendance decision, while reliability refers to the system's ability to perform consistently.

From an accuracy point of view, the system should correctly verify a student's identity, validate the student's location, confirm that the submitted QR token belongs to the active attendance session and prevent duplicate submissions. Since each individual verification method has limitations, the system should not depend on one single factor. GPS only attendance cannot be very accurate indoors and face verification alone cannot prove that the student is inside the class. Therefore, the selected design combines face verification, geofence validation, and dynamic QR to improve the accuracy of the attendance.

From a reliability perspective, the system should remain dependable during active lecture sessions, especially when many students attempt to mark attendance within a short time. The system will handle temporary network delays, poor image quality, GPS variation, expired QR tokens and repeated submissions without corrupting attendance records. Failed verification attempts should be logged, genuine students should be allowed to retry, and lecturers should have a controlled manual override process for valid exceptional cases.

#### 2. Security and Privacy

Security and privacy are major considerations because the system handles student accounts, attendance records, facial verification data, and location data. The system will use secure authentication, role based access control, encrypted communication, and strong password hashing to prevent unauthorized access.

Only the data required for attendance verification will be collected and stored. Face embeddings and attendance records will be accessible only to authorized users, and location data should be used only during attendance check-in, not for continuous student tracking, in order to ensure student privacy. The dynamic QR code should also be protected against misuse. QR tokens will be securely generated, short-lived, linked to the active attendance session, and validated using server-side time.

#### 3. Performance

Performance is an important consideration because attendance marking must be completed within a short time during an active lecture session. The system should minimize check-in delay, especially when many students attempt to mark attendance at the same time. Therefore, lightweight validations such as QR verification and geofence checking should be handled quickly, while face verification should be optimized using compressed images, preprocessing, and precomputed reference embeddings. Redis caching and controlled background processing [24] will be used during peak check-in periods to reduce repeated database access and prevent backend overload.

A video-based backend liveness approach was considered, but it can introduce performance bottlenecks due to large upload sizes, network dependency, and increased backend processing load. Therefore, basic liveness checks are planned to be handled on the mobile application, while backend processing focuses on face verification. This reduces network overhead and improves the overall responsiveness of the attendance marking process.

#### 4. Usability and Ease of Use

Usability is an important consideration because the system will be used frequently by students, lecturers, and administrators. The student check-in process should be simple and guided, with clear instructions for location validation, QR scanning, face capture, and attendance confirmation. The lecturer and administrator interfaces should be straightforward, allowing attendance sessions, reports, users, courses, classrooms, and geofence settings to be managed without unnecessary complexity. User manuals will also be provided to help each user understand the system functions and reduce difficulties during adoption.

#### 5. Maintainability

Maintainability is an important consideration because the system includes several connected functions such as authentication, attendance session management, QR validation, geofence validation, face verification, reporting, and audit logging. The system should be developed using a service-oriented modular architecture so that each functional module can be updated, tested, and maintained with minimal impact on unrelated parts of the system. Clear code organization, API documentation, database migrations, structured logging, and proper technical documentation should be maintained throughout the project. This will make the system easier to extend, debug, and deploy.

#### 6. Scalability

The system may need to support many students, courses, lecturers, and attendance sessions as it expands across departments, faculties, or other attendance-based environments such as training institutes and organizations. The system should be able to handle multiple attendance sessions, concurrent check-ins, growing attendance records, and increasing report generation requirements without significant degradation in performance. The proposed service-oriented architecture should support future growth through modular backend components, optimized database queries, proper indexing, caching for short-lived validation data, and asynchronous processing for non-critical tasks such as notifications, audit logs, and report updates. The face verification component should also be designed as a separate service module because its computational workload may require independent scaling when the number of concurrent check-ins increases.

#### 7. Availability and fault tolerance

Attendance marking is time sensitive and usually happens within a limited time period. The system should remain accessible during active attendance sessions and should handle failures such as network issues, GPS inaccuracies, expired QR tokens, poor image quality, and temporary backend errors without disrupting the entire attendance process. The system should provide retry options for students when verification fails due to technical reasons. In valid exceptional cases, lecturers should be able to use a controlled manual override process, where the reason for the override is recorded. This ensures that genuine students are not unfairly affected by temporary system or device-related failures.

#### 8. Auditability and Accountability

Attendance records can affect academic decisions. Therefore, the system should maintain database audit logs for important actions such as attendance session creation, QR validation results, face verification results, manual overrides and administrative changes.

As described in the mitigation for incorrect manual approval by a lecturer, the system will record the lecturer who performed a manual override, the affected student, the related

attendance session, the previous attendance status, the updated status, the reason for the change, and the timestamp. These audit logs will help maintain transparency, prevent misuse, and support dispute resolution when attendance records are questioned.

#### 4. References

- [1] Jibble, “Free Attendance Management Software,” Jibble. [Online]. Available: <https://www.jibble.io/attendance-management>. [Accessed Jun. 28, 2026].
- [2] Truein, “Geofencing Attendance System | GPS Based Attendance App for Field Staff,” Truein. [Online]. Available: <https://truein.com/geofencing-attendance-system>. [Accessed Jun. 28, 2026].
- [3] Qwickly, “Qwickly Attendance Mobile App Setup,” Qwickly Help Center. [Online]. Available: <https://qwickly.zendesk.com/hc/en-us/articles/13295304911501-Qwickly-Attendance-Mobile-App-Setup>. [Accessed Jun. 28, 2026].
- [4] SEAtS, “Student Attendance Management Made Easy Today,” SEAtS ONE. [Online]. Available: <https://seatsone.ai/student-attendance-management/>. [Accessed Jun. 28, 2026].
- [5] A. Rao, “AttenFace: A Real Time Attendance System using Face Recognition,” arXiv preprint arXiv:2211.07582, Nov. 2022. [Online]. Available: <https://arxiv.org/abs/2211.07582>.
- [6] L. K. Babu, N.V. Prasad, S. Yeswanth and E. P. Kumar, “Smart Attendance System using Geofencing and Facial Recognition”, Int. J. Progressive Res. Eng. Manage. Sci. (IJPREMS), vol. 05, no. 11, pp. 291-297, Nov. 2025. [Online]. Available: <https://www.ijprems.com/ijprems-paper/smart-attendance-system-using-geofencing-and-face-recognition>.
- [7] Vercel, “Next.js Documentation,” Next.js. [Online]. Available: <https://nextjs.org/docs>. [Accessed Jun. 27, 2026].
- [8] Meta Platforms, Inc., “Get Started Without a Framework,” React Native Documentation.” [Online]. Available: <https://reactnative.dev/docs/getting-started-without-a-framework>. [Accessed Jun. 27, 2026].
- [9] S. Ramírez, “FastAPI,” FastAPI Documentation.” [Online]. Available: <https://fastapi.tiangolo.com/>. [Accessed Jun. 27, 2026].
- [10] PostgreSQL Global Development Group, “PostgreSQL Documentation,” PostgreSQL. [Online]. Available: <https://www.postgresql.org/docs/current/>. [Accessed Jun. 27, 2026].
- [11] S. I. Serengil, “DeepFace: A Lightweight Face Recognition and Facial Attribute Analysis Library for Python,” GitHub. [Online]. Available: <https://github.com/serengil/deepface>. [Accessed Jun. 28, 2026].
- [12] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019.

- [13] Scikit-learn Developers, “cosine\_similarity,” Scikit-learn Documentation, version 1.9.0. [Online]. Available: [https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine\\_similarity.html](https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html). [Accessed Jun. 28, 2026].
- [14] Google, “Face detection | ML Kit | Google for Developers,” Google for Developers. [Online]. Available: <https://developers.google.com/ml-kit/vision/face-detection>. [Accessed Jun. 30, 2026].
- [15] OpenCV, “OpenCV Documentation,” OpenCV. [Online]. Available: <https://docs.opencv.org>. [Accessed Jun. 28, 2026].
- [16] Amazon Web Services, “AWS Free Tier,” AWS. [Online]. Available: <https://aws.amazon.com/free/>. [Accessed Jul. 11, 2026].
- [17] Amazon Web Services, “Choosing an AWS Free Tier Plan,” AWS Billing Documentation. [Online]. Available: <https://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/free-tier-plans.html>. [Accessed Jul. 11, 2026].
- [18] Supabase, “Pricing & Fees,” Supabase. [Online]. Available: <https://supabase.com/pricing>. [Accessed Jul. 8, 2026].
- [19] Supabase, “Database Backups,” Supabase Docs. [Online]. Available: <https://supabase.com/docs/guides/platform/backups>. [Accessed Jul. 8, 2026].
- [20] Cloudflare, “R2 Pricing,” Cloudflare Developers. [Online]. Available: <https://developers.cloudflare.com/r2/pricing/>. [Accessed Jul. 11, 2026].
- [21] Vercel, “Vercel Hobby Plan,” Vercel Documentation. [Online]. Available: <https://vercel.com/docs/plans/hobby>. [Accessed Jul. 11, 2026].
- [22] Expo, “Expo Application Services Pricing,” Expo. [Online]. Available: <https://expo.dev/pricing>. [Accessed Jul. 11, 2026].
- [23] Expo, “Expo Documentation,” Expo Documentation. [Online]. Available: <https://docs.expo.dev/>. [Accessed Jun. 28, 2026].
- [24] GeeksforGeeks, “Redis Cache,” GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/system-design/redis-cache/>. [Accessed Jul. 10, 2026].
- [25] Redis, “Redis Rate Limiter,” Redis Documentation. [Online]. Available: <https://redis.io/docs/latest/develop/use-cases/rate-limiter/>. [Accessed Jul. 10, 2026].
- [26] Redis, “Redis Job Queue,” Redis Documentation. [Online]. Available: <https://redis.io/docs/latest/develop/use-cases/job-queue/>. [Accessed Jul. 10, 2026].